

Empirical Security & Privacy, for Humans

UPenn CIS 7000-010

11/13/2025

The Matter of
Heartbleed



Reading

The Matter of Heartbleed

*Zakir Durumeric¹, James Kasten¹,
David Adrian¹, J. Alex Halderman¹,
Michael Bailey^{1,2}

¹ University of Michigan
² University of Illinois, Urbana Champaign
{zakir, jdkasten, davadria, jhalderm}@umich.edu,
mdbailey@illinois.edu

*Frank Li³, Nicholas Weaver^{3,4},
Johanna Amann⁴, Jethro Beekman³,
Mathias Payer^{3,5}, Vern Paxson^{3,4}

³ EECS, University of California, Berkeley
⁴ International Computer Science Institute
⁵ Purdue University
{frankli, nweaver, jbeekman, vern}@cs.berkeley.edu,
johanna@icir.org, mpayer@purdue.edu

ABSTRACT

The Heartbleed vulnerability took the Internet by surprise in April 2014. The vulnerability, one of the most consequential since the advent of the commercial Internet, allowed attackers to remotely read protected memory from an estimated 24–55% of popular HTTPS sites. In this work, we perform a comprehensive, measurement-based analysis of the vulnerability's impact, including (1) tracking the vulnerable population, (2) monitoring patching behavior over time, (3) assessing the impact on the HTTPS certificate ecosystem, and (4) exposing real attacks that attempted to exploit the bug. Furthermore, we conduct a large-scale vulnerability notification experiment involving 150,000 hosts and observe a nearly 50% increase in patching by notified hosts. Drawing upon these analyses, we discuss what went well and what went poorly, in an effort to understand how the technical community can respond more effectively to such events in the future.

1. INTRODUCTION

In March 2014, researchers found a catastrophic vulnerability in OpenSSL, the cryptographic library used to secure connections in popular server products including Apache and Nginx. While OpenSSL has had several notable security issues during its 16 year history, this flaw—the *Heartbleed* vulnerability—was one of the most impactful. Heartbleed allows attackers to read sensitive memory from vulnerable servers, potentially including cryptographic keys, login credentials, and other private data. Exacerbating its severity, the bug is simple to understand and exploit.

In this work, we analyze the impact of the vulnerability and track the server operator community's responses. Using extensive active scanning, we assess who was vulnerable, characterizing Heartbleed's scope across popular HTTPS websites and the full IPv4 address space. We also survey the range of protocols and server products affected. We estimate that 24–55% of HTTPS servers in the Alexa Top 1 Million were initially vulnerable, including 44 of

*These authors contributed equally to this work.

the Alexa Top 100. Two days after disclosure, we observed that 11% of HTTPS sites in the Alexa Top 1 Million remained vulnerable, as did 6% of all HTTPS servers in the public IPv4 address space. We find that vulnerable hosts were not randomly distributed, with more than 50% located in only 10 ASes that do not reflect the ASes with the most HTTPS hosts. In our scans of the IPv4 address space, we identify over 70 models of vulnerable embedded devices and software packages. We also observe that both SMTP+TLS and Tor were heavily affected; more than half of all Tor nodes were vulnerable in the days following disclosure.

Our investigation of the operator community's response finds that within the first 24 hours, all but 5 of the Alexa Top 100 sites were patched, and within 48 hours, all of the vulnerable hosts in the top 500 were patched. While popular sites responded quickly, we observe that patching plateaued after about two weeks, and 3% of HTTPS sites in the Alexa Top 1 Million remained vulnerable almost two months after disclosure.

In addition to patching, many sites replaced their TLS certificates due to the possibility that the private keys could have been leaked. We analyze certificate replacement and find that while many of the most popular websites reacted quickly, less than a quarter of Alexa Top 1 Million sites replaced certificates in the week following disclosure. Even more worryingly, only 10% of the sites that were vulnerable 48 hours after disclosure replaced their certificates within the next month, and of those that did, 14% neglected to change the private key, gaining no protection from certificate replacement.

We also investigate widespread attempts to exploit Heartbleed, as seen in extensive bulk traffic traces recorded at four sites. We find no evidence of exploitation prior to the vulnerability's public disclosure, but we detect subsequent exploit attempts from almost 700 sources, beginning less than 24 hours after disclosure. Comparing attack attempts across sites, we observe that despite the large number of sources and scans, only a handful appear to reflect exhaustive Internet-wide scans.

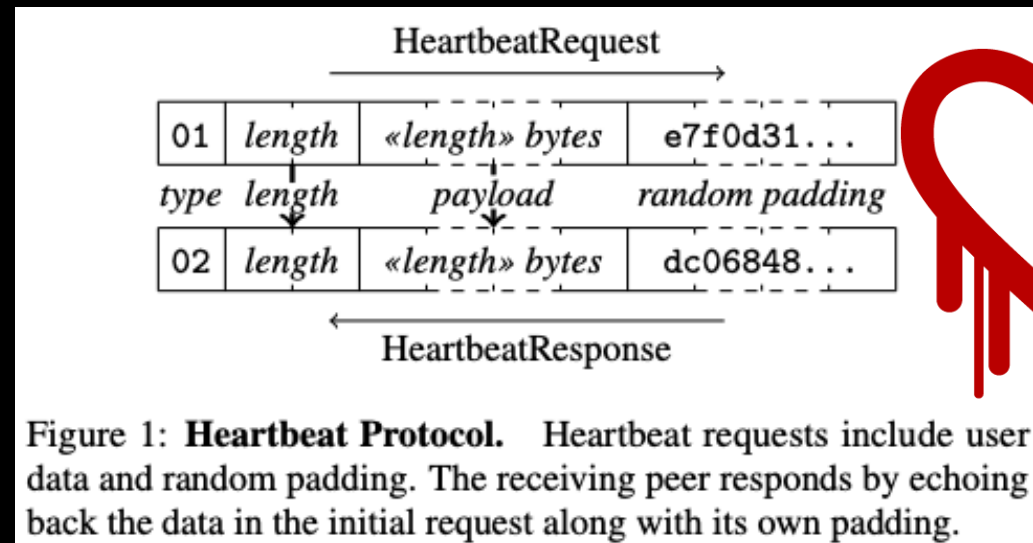
Finally, starting three weeks after disclosure, we undertook a large-scale notification effort and contacted the operators responsible for the remaining vulnerable servers. By contacting the operators in two waves, we could conduct a controlled experiment and measure the impact of notification on patching. We report the effects of our notifications, observing a surprisingly high 47% increase in patching by notified operators.

We draw upon these observations to discuss both what went well and what went poorly in the aftermath of Heartbleed. By better understanding the lessons of this security disaster, the technical community can respond more effectively to such events in the future.



OpenSSL and the Heartbleed vulnerability

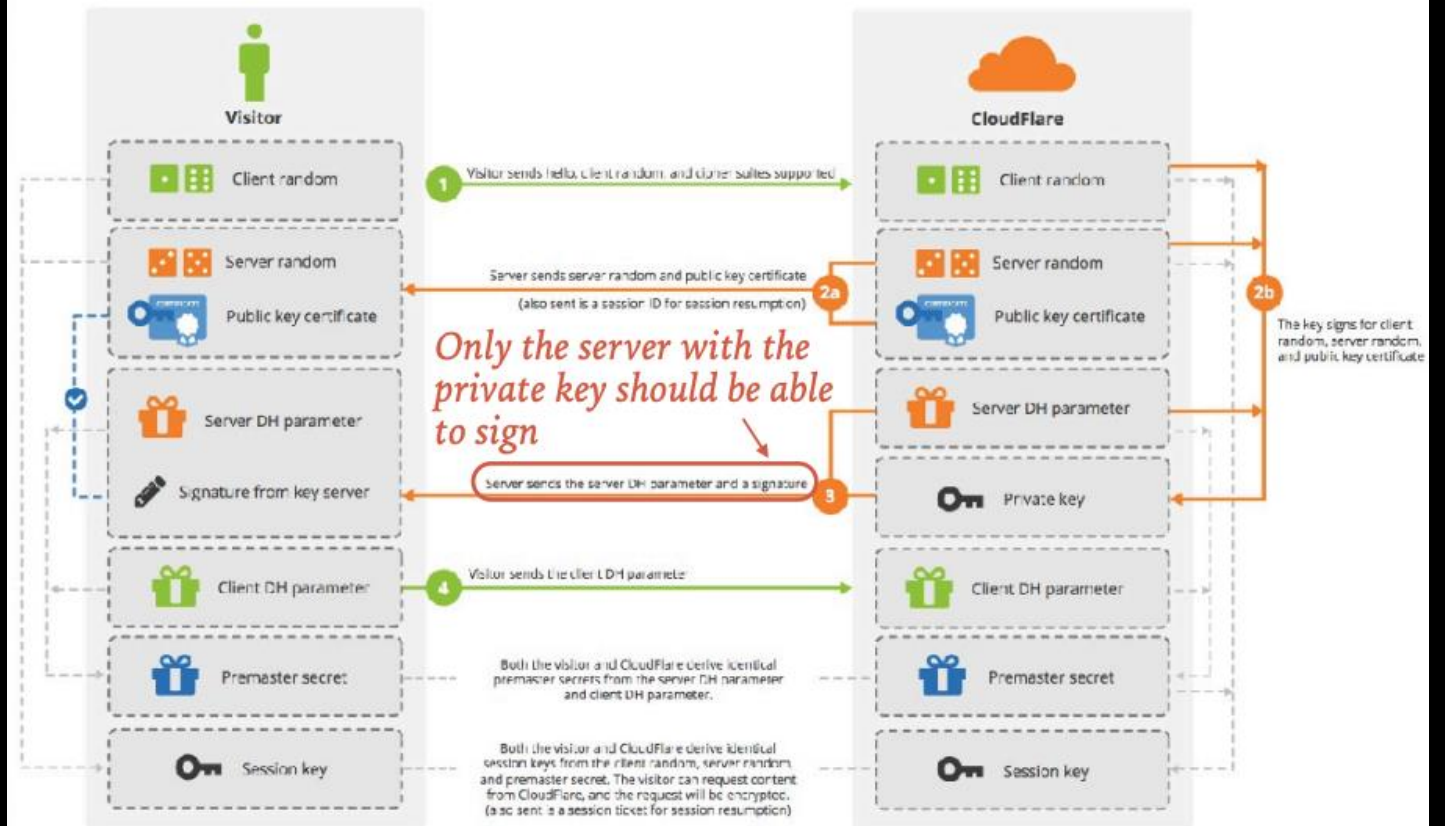
- OpenSSL is a popular open-source cryptographic library, started in 1998, that implements the SSL and TLS protocols. Widely used.
- Heartbeat: Server echoes back N bytes provided in the client packet, where N is the given length
- Bug: Server trusts N !
If $N >$ actual packet size, server will read (“bleed”) and return its own memory
- Risk: Bled memory could contain secrets like cryptographic keys
 - And: Easy to exploit the bug!



How are keys
used with
TLS?

SSL Handshake (Diffie-Hellman)

Handshake



(Credit: CloudFlare)

Verifying certificates



"I'm  because I say so!"



"I'm  because  says so"

Certificates like these
issued by Certificate
Authorities (CAs)



"I'm  because  says so"

Verifying certificates



"I'm ✓ because I say so!"

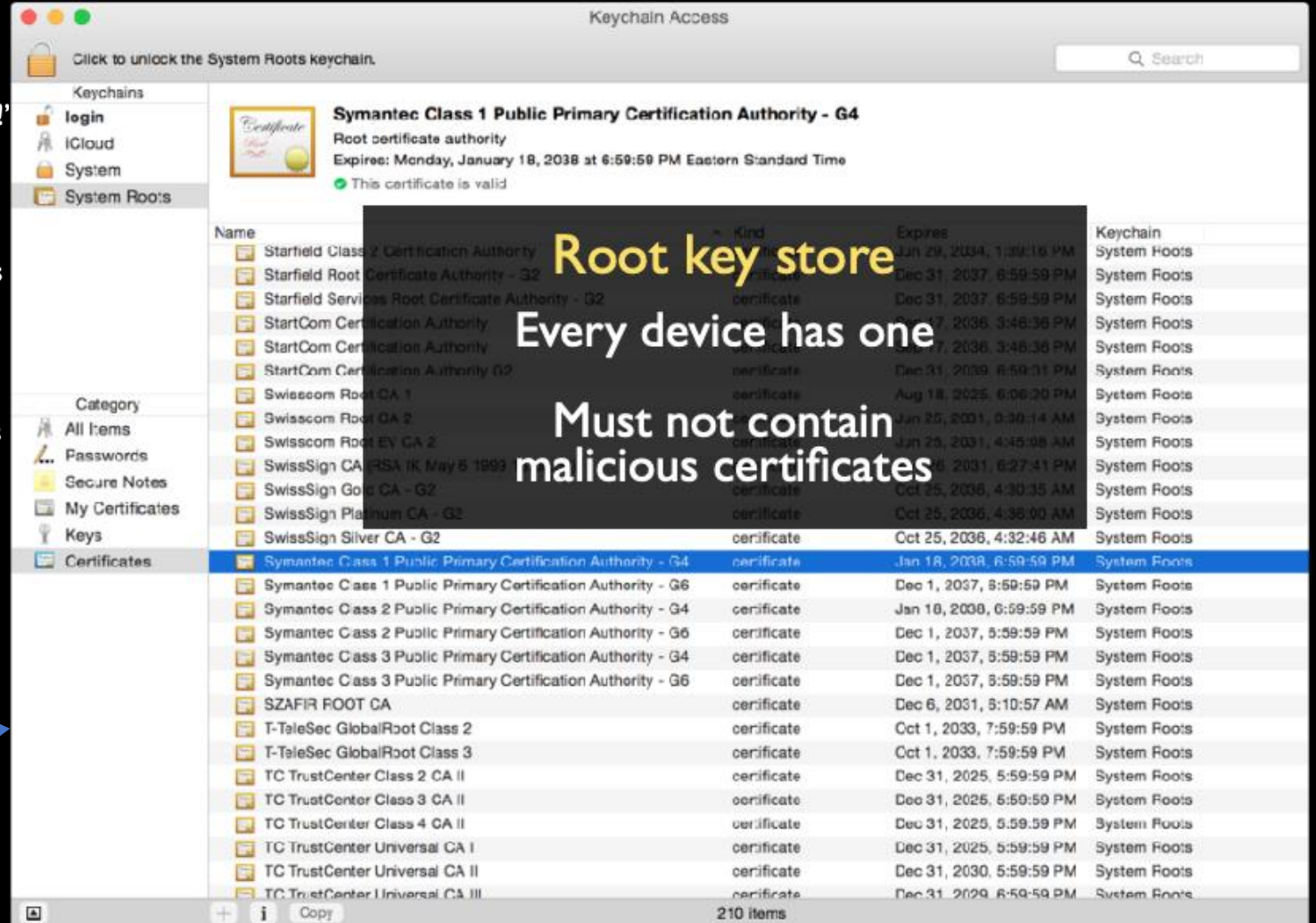


"I'm ✓ because ✓ says



"I'm → because ✓ says

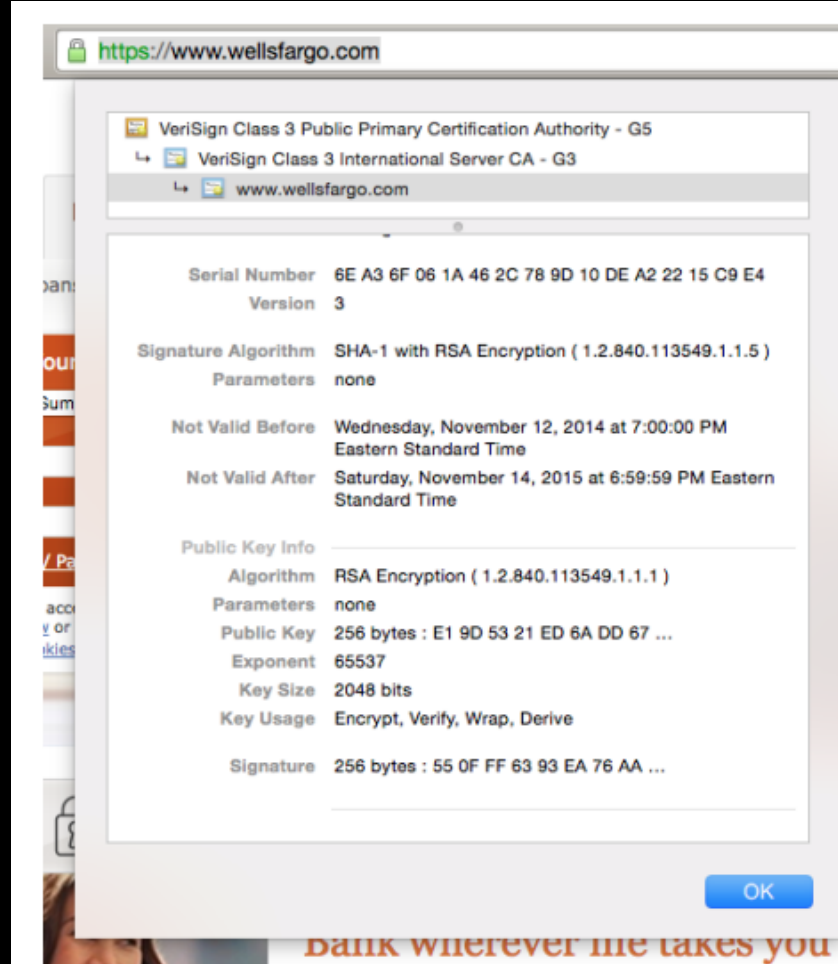
Certificate
authorities
included here



Verifying certificates



Browser



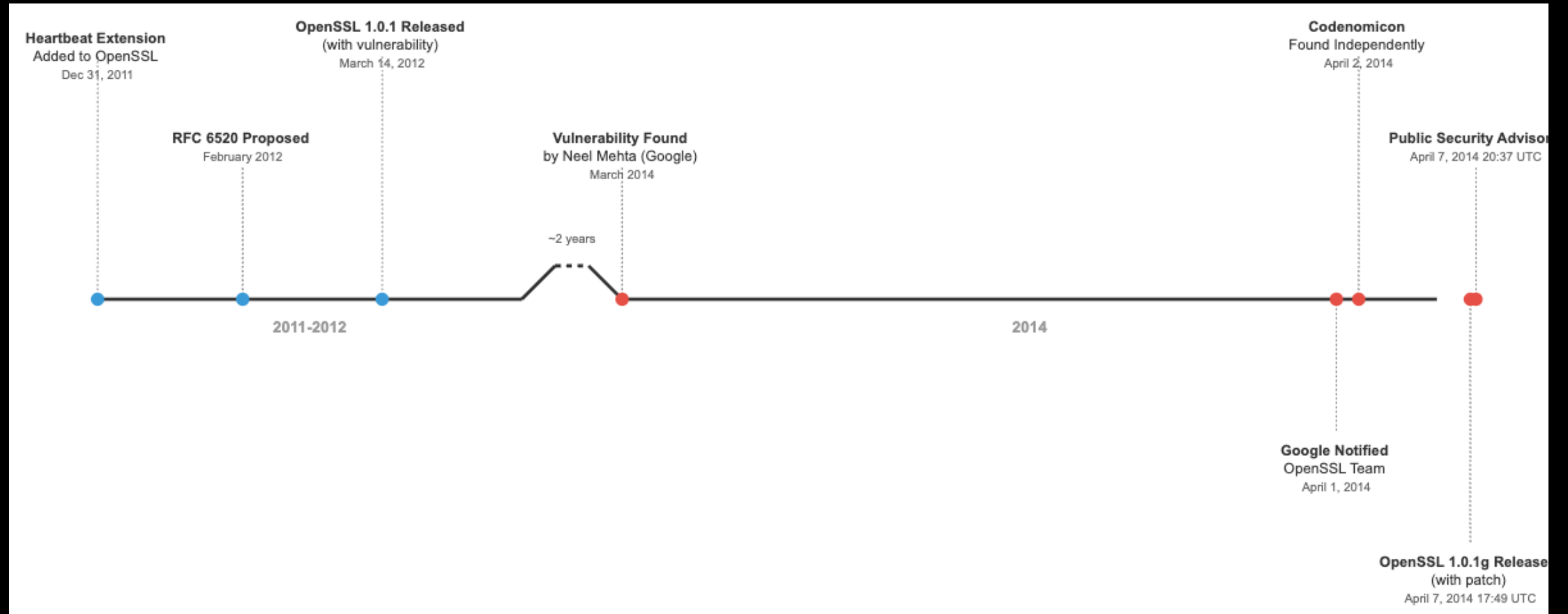
Serial number: Uniquely identifies this cert with respect to the issuer (look for this in CRLs)

Signature algorithm: How the issuer will sign parts of the cert

Not valid before/after: When to start and stop believing this cert (start & expiration dates)

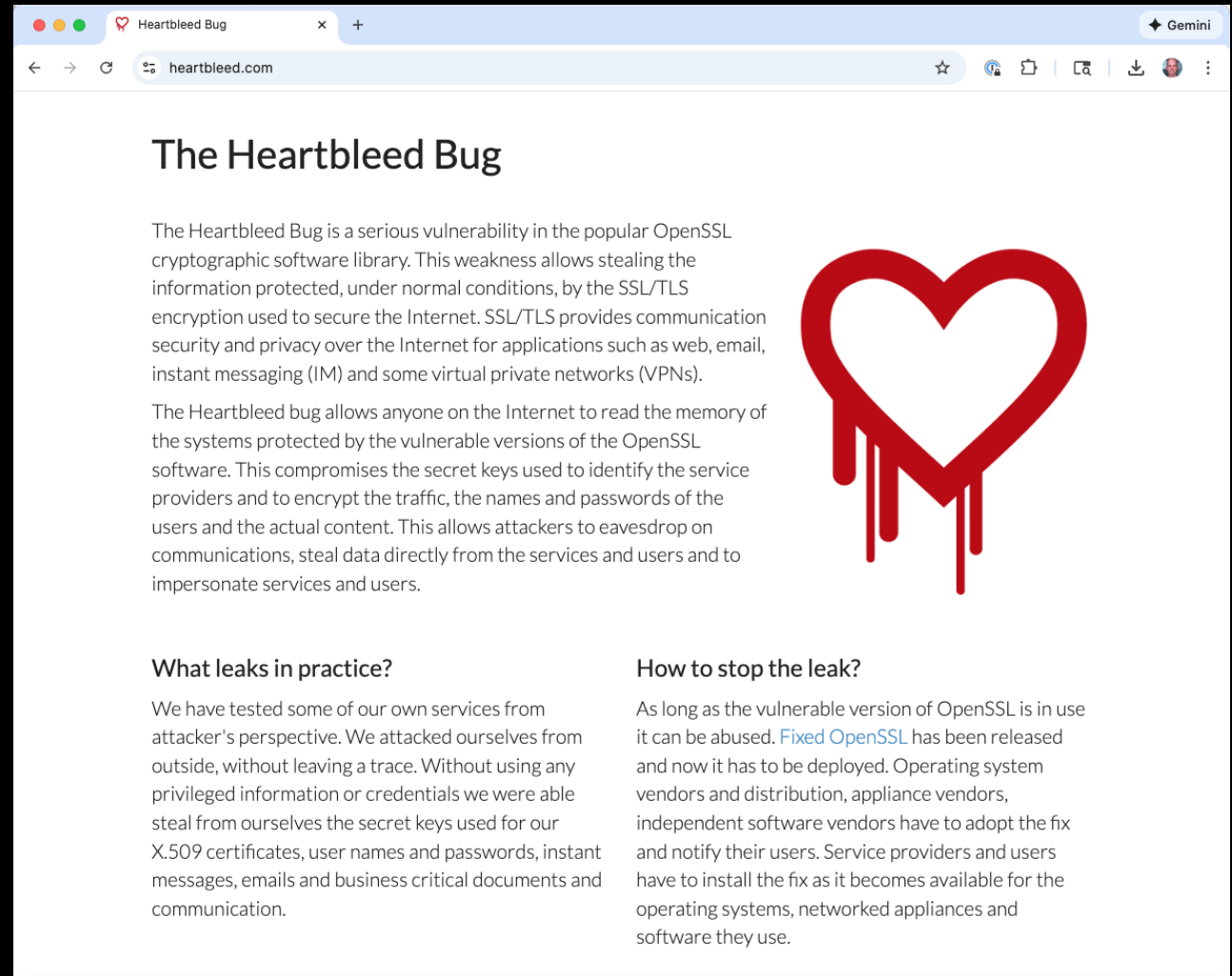
The public key: And the issuer's signature of the public key

Heartbleed timeline



Recommended responses to Heartbleed

1. Patch the server
2. “Reissue” a new public key (get a new one and load it on your servers)
3. Revoke the old key

A screenshot of a web browser displaying the 'Heartbleed Bug' website. The browser's address bar shows 'heartbleed.com'. The page has a white background with black text. The title 'The Heartbleed Bug' is at the top. Below it, a paragraph explains the vulnerability in OpenSSL. To the right of this paragraph is a red heart icon with red liquid dripping down from its base. Below the main text, there are two columns: 'What leaks in practice?' and 'How to stop the leak?'. The first column describes how the bug allows attackers to steal sensitive data like passwords and private keys. The second column explains that the fix is to update to the latest version of OpenSSL and that various vendors are responsible for deploying the patch.

The Heartbleed Bug

The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software library. This weakness allows stealing the information protected, under normal conditions, by the SSL/TLS encryption used to secure the Internet. SSL/TLS provides communication security and privacy over the Internet for applications such as web, email, instant messaging (IM) and some virtual private networks (VPNs).

The Heartbleed bug allows anyone on the Internet to read the memory of the systems protected by the vulnerable versions of the OpenSSL software. This compromises the secret keys used to identify the service providers and to encrypt the traffic, the names and passwords of the users and the actual content. This allows attackers to eavesdrop on communications, steal data directly from the services and users and to impersonate services and users.



What leaks in practice?

We have tested some of our own services from attacker's perspective. We attacked ourselves from outside, without leaving a trace. Without using any privileged information or credentials we were able to steal from ourselves the secret keys used for our X.509 certificates, user names and passwords, instant messages, emails and business critical documents and communication.

How to stop the leak?

As long as the vulnerable version of OpenSSL is in use it can be abused. [Fixed OpenSSL](#) has been released and now it has to be deployed. Operating system vendors and distribution, appliance vendors, independent software vendors have to adopt the fix and notify their users. Service providers and users have to install the fix as it becomes available for the operating systems, networked appliances and software they use.

So what happened?

Research questions

- Which software was vulnerable?
- Which sites were vulnerable?
- When were they patched?
- When were certificates replaced?
- Was there evidence of attempted exploitation?
- Did patching rates improve after a notification campaign?

Primary method: ZMap scans

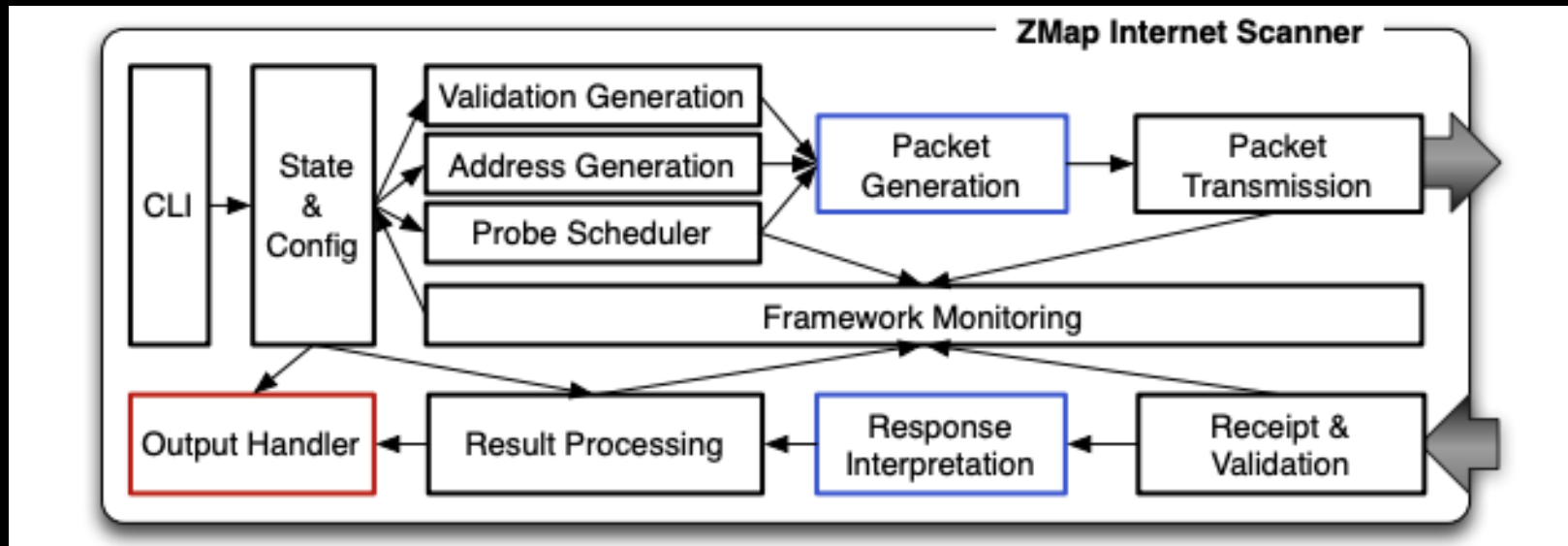
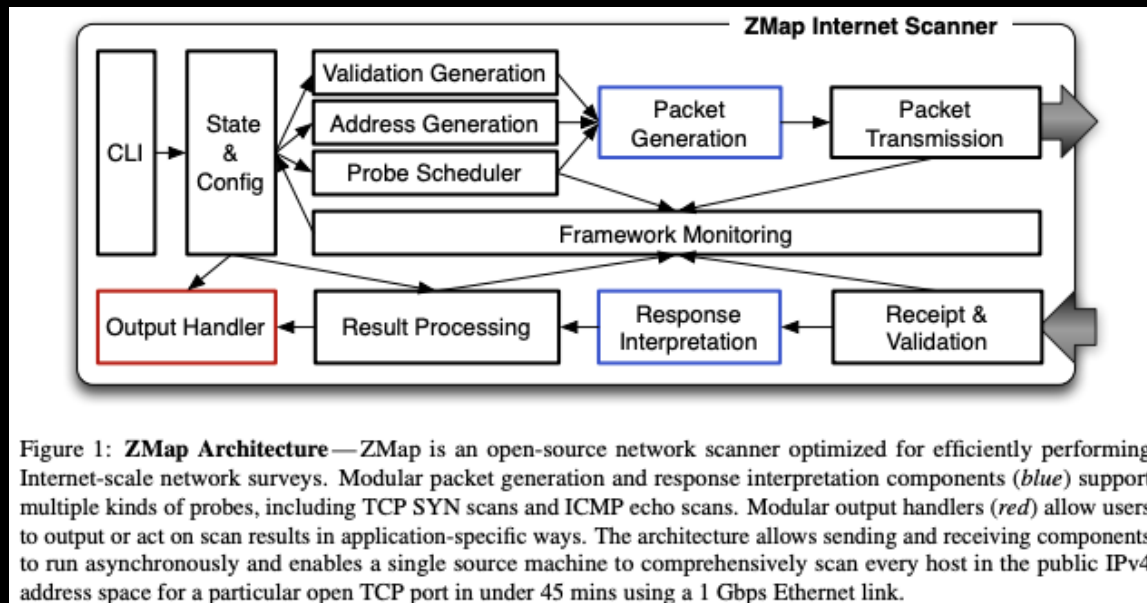


Figure 1: **ZMap Architecture**—ZMap is an open-source network scanner optimized for efficiently performing Internet-scale network surveys. Modular packet generation and response interpretation components (*blue*) support multiple kinds of probes, including TCP SYN scans and ICMP echo scans. Modular output handlers (*red*) allow users to output or act on scan results in application-specific ways. The architecture allows sending and receiving components to run asynchronously and enables a single source machine to comprehensively scan every host in the public IPv4 address space for a particular open TCP port in under 45 mins using a 1 Gbps Ethernet link.

Durumeric, Wustrow, and Halderman: *ZMap: Fast Internet-Wide Scanning and its Security Applications*, USENIX Security 2013

Primary method: ZMap scans



Durumeric, Wustrow, and Halderman: *ZMap: Fast Internet-Wide Scanning and its Security Applications*, USENIX Security 2013



- Connect to host via TLS
- Send Heartbeat message with size 0
- If response received, then vulnerable

Analysis of the HTTPS Certificate Ecosystem*

Zakir Durumeric, James Kasten, Michael Bailey, J. Alex Halderman
Department of Electrical Engineering and Computer Science
University of Michigan, Ann Arbor, MI 48109, USA
{zakir, jdkasten, mibailey, jhalderm}@umich.edu

ABSTRACT

We report the results of a large-scale measurement study of the HTTPS certificate ecosystem—the public-key infrastructure that underlies nearly all secure web communications. Using data collected by performing 110 Internet-wide scans over 14 months, we gain detailed and temporally fine-grained visibility into this otherwise opaque area of security-critical infrastructure. We investigate the trust relationships among root authorities, intermediate authorities, and the leaf certificates used by web servers, ultimately identifying and classifying more than 1,800 entities that are able to issue certificates vouching for the identity of any website. We uncover practices that may put the security of the ecosystem at risk, and we identify frequent configuration problems that lead to user-facing errors and potential vulnerabilities. We conclude with lessons and recommendations to ensure the long-term health and security of the certificate ecosystem.

Categories and Subject Descriptors

C.2.2 [Computer-Communication Networks]: [Network Protocols]; C.2.3 [Computer-Communication Networks]: [Network Operations]; E.3 [Data Encryption]: [Public key cryptosystems, Standards]

Keywords

TLS; SSL; HTTPS; public-key infrastructure; X.509; certificates; security; measurement; Internet-wide scanning

1. INTRODUCTION

Nearly all secure web communication takes place over HTTPS including online banking, e-mail, and e-commerce transactions. HTTPS is based on the TLS encrypted transport protocol and a

supporting public key infrastructure (PKI) composed of thousands of certificate authorities (CAs)—entities that are trusted by users' browsers to vouch for the identity of web servers. CAs do this by signing digital certificates that associate a site's public key with its domain name. We place our full trust in each of these CAs—in general, every CA has the ability to sign trusted certificates for *any* domain, and so the entire PKI is only as secure as the weakest CA. Nevertheless, this complex distributed infrastructure is strikingly opaque. There is no published list of signed website certificates or even of the organizations that have trusted signing ability. In this work, we attempt to rectify this and shed light on the HTTPS certificate ecosystem.

Our study is founded on what is, to the best of our knowledge, the most comprehensive dataset of the HTTPS ecosystem to date. Between June 2012 and August 2013, we completed 110 exhaustive scans of the public IPv4 address space in which we performed TLS handshakes with all hosts publicly serving HTTPS on port 443. Over the course of 14 months, we completed upwards of 400 billion SYN probes and 2.55 billion TLS handshakes, collecting and parsing 42.4 million unique X.509 certificates from 109 million hosts. On average, each of our scans included 178% more TLS hosts and 115% more certificates than were collected in earlier studies of the certificate authority ecosystem [14], and we collected 736% more unique certificates in total than any prior study of HTTPS [16].

Using this dataset, we investigate two classes of important security questions, which relate to the behavior of CAs and to site certificates.

Certificate Authorities We analyze the organizations involved in the HTTPS ecosystem and identify 1,832 CA certificates, which are controlled by 683 organizations including religious institutions, museums, libraries, and more than 130 corporations and financial institutions. We find that more than 80% of the organizations with a signing certificate are not commercial certificate authorities and further investigate the paths through which organizations are acquiring signing certificates. We investigate the constraints on these CA certificates and find that only 7 CA certificates use name constraints, and more than 40% of CA certificates have no path length constraint. We identify two sets of misissued CA certificates and discuss their impact on the security of the ecosystem.

*Permission to make digital or hard copies of part or all of this work is granted without fee provided that copies are not distributed for profit and that copies bear this notice and the full citation on the first page. Copyright is held by the authors. This more permissive statement supersedes the ACM-mandated statement below, which

Which software was vulnerable?

Used TLS implemented by OpenSSL:

Web Servers		Mail Servers		Database Servers		XMPP Servers		Other Servers	
Apache (mod_ssl) [45]	Yes	Sendmail [62]	Yes	MySQL [62]	Yes	OpenFire [12]	No	OpenVPN [54]	Yes
Microsoft IIS [46]	No	Postfix [62]	Yes	PostgreSQL [62]	Yes	Ejabberd [5]	Yes	OpenLDAP [63]	Yes
Nginx [14]	Yes	Qmail [62]	Yes	SQL Server [46]	No	Jabberd14 [70]	Yes	Stunnel [65]	Yes
Lighttpd [62]	Yes	Exim [35]	Yes	Oracle [55]	No	Jabberd2 [41]	Yes	Openswan [49]	Yes
Tomcat [17]	Yes	Courier [37]	Yes	IBM DB2 [38]	No			Telnetd-ssl [4]	Yes
Google GWS [50]	Yes	Exchange [46]	No	MongoDB [47]	Yes			OpenDKIM [3]	Yes
LiteSpeed [42]	Yes	Dovecot [35]	Yes	CouchDB [32]	No			Proftpd [64]	Yes
IBM Web Server [38]	Yes	Cyrus [48]	Yes	Cassandra [6]	No			Bitcoin Client [24]	Yes
Tengine [13]	Yes	Zimbra [56]	Yes	Redis [35]	No				
Jetty [51]	No								

Table 2: **Vulnerable Server Products.** We survey which server products were affected by Heartbleed.

Which sites were vulnerable?

Site	Vuln.	Site	Vuln.	Site	Vuln.
Google	Yes	Bing	No	Wordpress	Yes
Facebook	No	Pinterest	Yes	Huff. Post	?
Youtube	Yes	Blogspot	Yes	ESPN	?
Yahoo	Yes	Go.com	?	Reddit	Yes
Amazon	No	Live	No	Netflix	Yes
Wikipedia	Yes	CNN	?	MSN.com	No
LinkedIn	No	Instagram	Yes	Weather.com	?
eBay	No	Paypal	No	IMDB	No
Twitter	No	Tumblr	Yes	Apple	No
Craigslist	?	Imgur	Yes	Yelp	?

Table 3: Vulnerability of Top 30 US HTTPS-Enabled Websites. We aggregate published lists of vulnerable sites, press releases, and news sources to determine which of the top sites were vulnerable before the discovery of Heartbleed.

All of the Alexa Top 100 websites were patched within 48 hours of disclosure—prior to the start of the authors' scans

11.5% of HTTPS-enabled sites were vulnerable at their first scan. Between 24–55% of the *Alexa Top 1 Million* HTTPS websites vulnerable at the time of disclosure

Web Server	Alexa Sites	Heartbeat Ext.	Vulnerable
Apache	451,270 (47.3%)	95,217 (58.4%)	28,548 (64.4%)
Nginx	182,379 (19.1%)	46,450 (28.5%)	11,185 (25.2%)
Microsoft IIS	96,259 (10.1%)	637 (0.4%)	195 (0.4%)
Litespeed	17,597 (1.8%)	6,838 (4.2%)	1,601 (3.6%)
Other	76,817 (8.1%)	5,383 (3.3%)	962 (2.2%)
Unknown	129,006 (13.5%)	8,545 (5.2%)	1,833 (4.1%)

Table 4: Alexa Top 1 Million Web Servers. We classify the web servers used by the Alexa Top 1 Million Sites, as observed in our first scan on April 9, 2014. Percentages represent the breakdown of server products for each category. We note that while Microsoft IIS was not vulnerable to Heartbleed, a small number of IIS servers used vulnerable SSL terminators.

When were they patched?

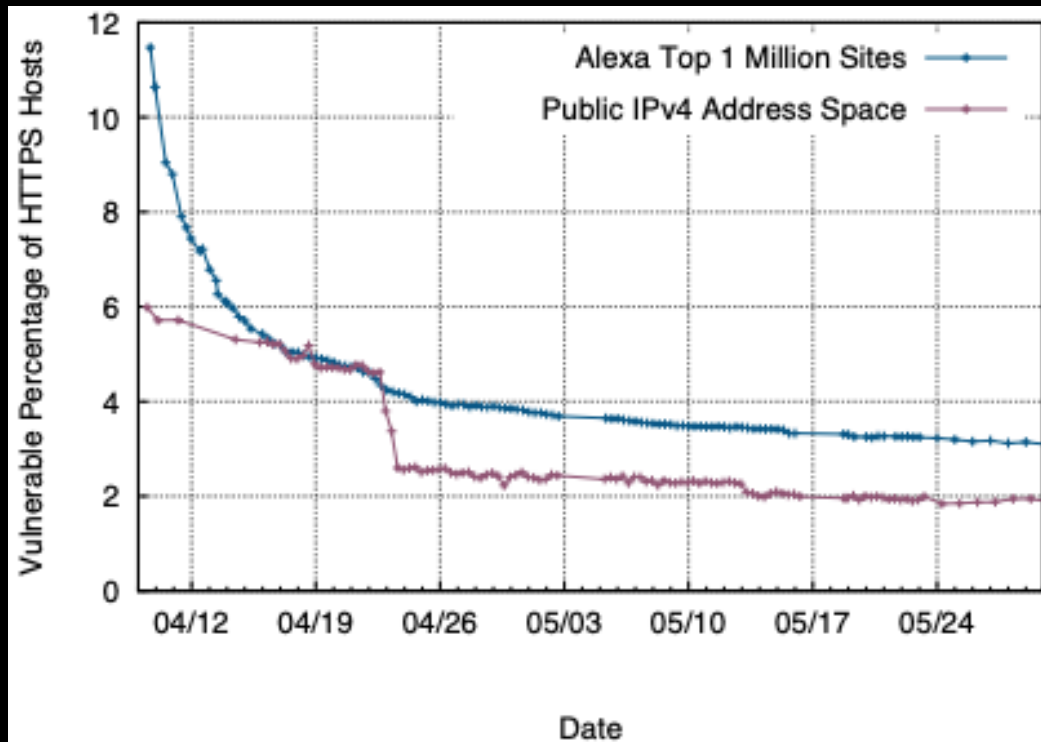


Figure 3: **HTTPS Patch Rate.** We track vulnerable web servers in the Alexa Top 1 Million and the public IPv4 address space. We track the latter by scanning independent 1% samples of the public IPv4 address space every 8 hours. Between April 9 and June 4, the vulnerable population of the Alexa Top 1 Million shrank from 11.5% to 3.1%, and for all HTTPS hosts from 6.0% to 1.9%.

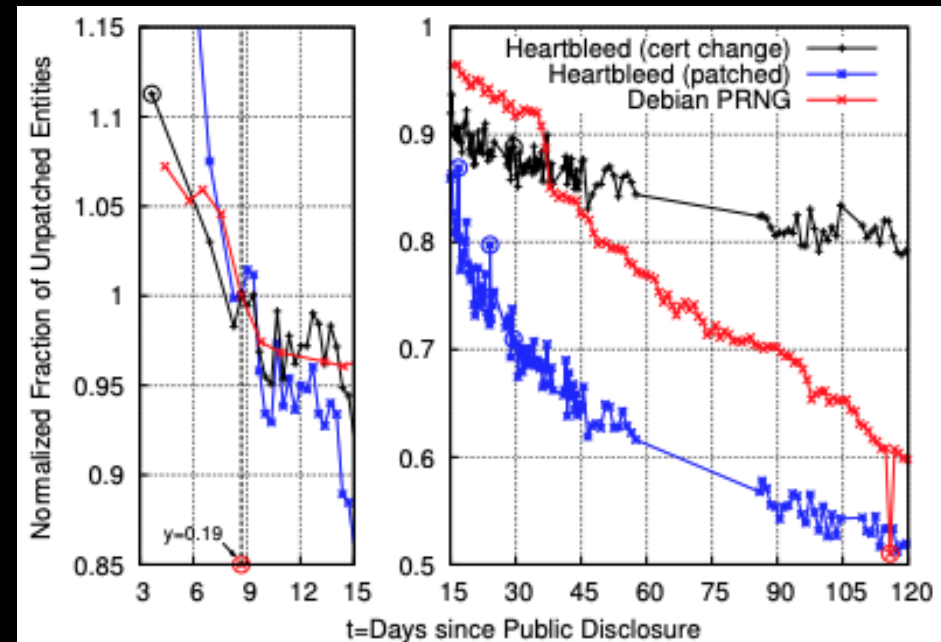


Figure 4: **Comparison of the Patch Rates of the Debian PRNG and Heartbleed Vulnerabilities.** The y-axis is normalized at 8.7 days, indicated by the vertical striped line. Thus, the fraction of unpatched entities at a given time is relative to the fraction at 8.7 days after disclosure, for each dataset. Except for the points marked by $\circ$, for each measurement the size of the Debian PRNG entity population was $n = 41,200 \pm 2,000$, and for Heartbleed, $n = 100,900 \pm 7,500$. Due to a misconfiguration in our measurement setup, no Heartbleed data is available days 58–85.

When were certificates reissued and revoked?

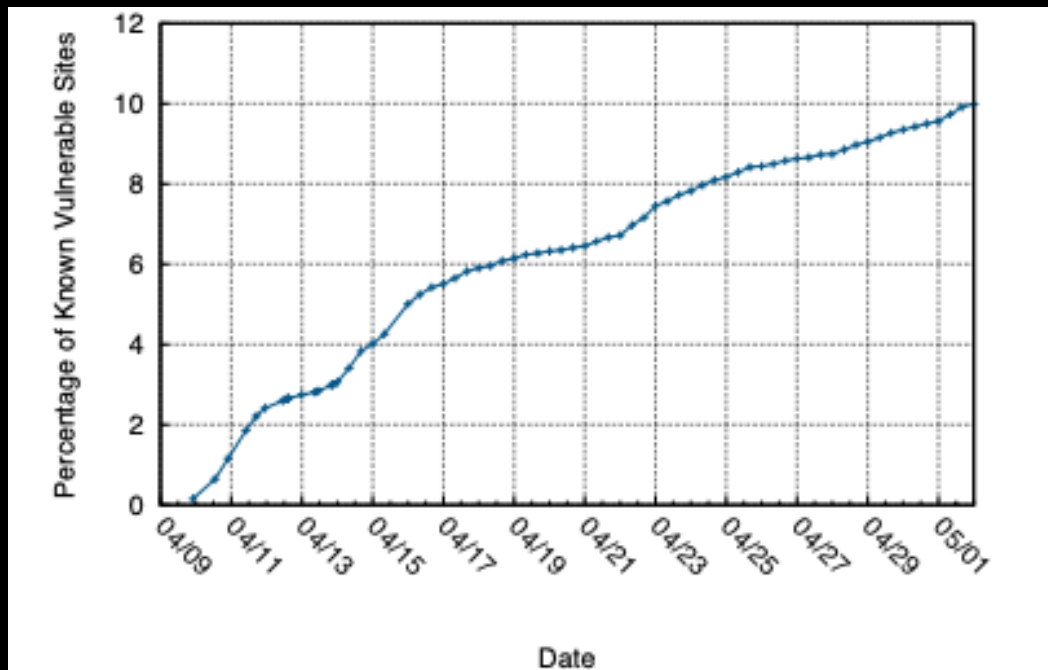


Figure 5: **Certificate Replacement on Vulnerable Alexa Sites.** We monitored certificate replacement on vulnerable Alexa Top 1 Million sites and observe only 10% replaced certificates in the month following public disclosure.

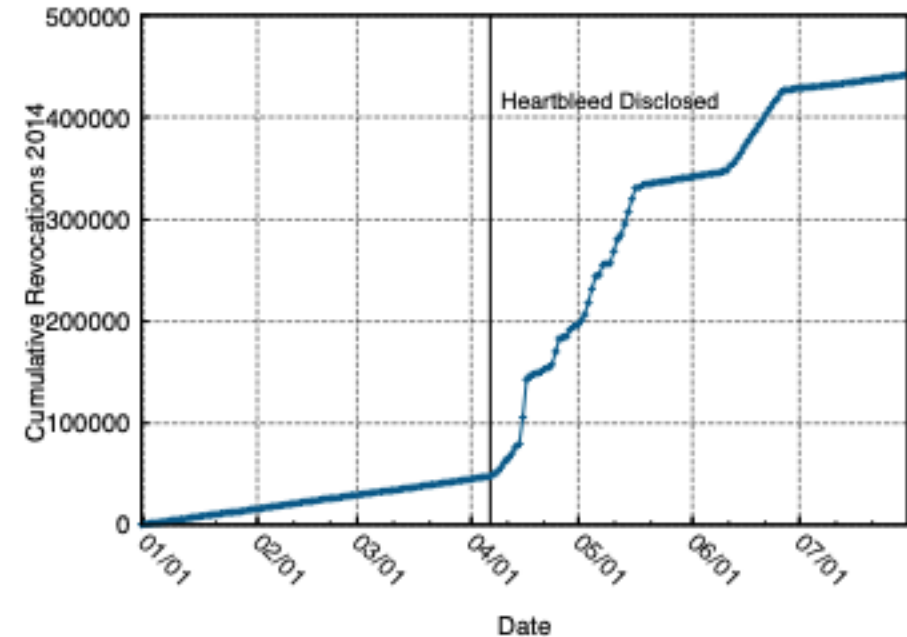


Figure 7: **Revocations after Heartbleed.** Certificate revocations dramatically increased after the disclosure. The jumps reflect GlobalSign (first) and GoDaddy (rest). However, still only 4% of HTTPS sites in the Alexa Top 1 Million revoked their certificates between April 9 and April 30, 2014.

Was there evidence of attempted exploitation?

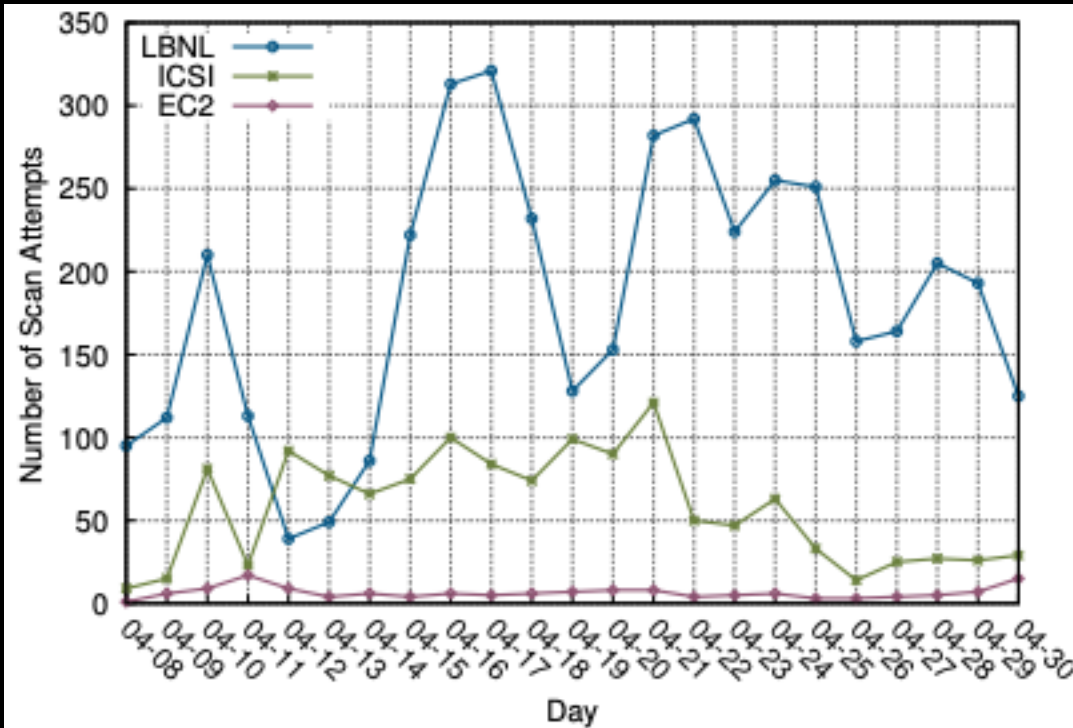


Figure 8: **Incoming Attacks.** We track the number of incoming connections containing Heartbleed exploit messages seen at ICSI, LBNL, and the EC2 honeypot per day.

Type	Sources	Targets	Connections
(1) Cleartext attacks	628	191	1,691
(2) Successful cleartext attacks	4	10	26
(3) Short Heartbeats	8	10	187
(4) Heartbeats before data	61	132	4,070
(5) Successful attacks after encryption	5	6	77
Total	692	217	5,948

Table 6: **Attack Types.** We list different stages of attacks observed against LBNL, ICSI and the EC2 honeypot.

Did patching rates improve after notification?

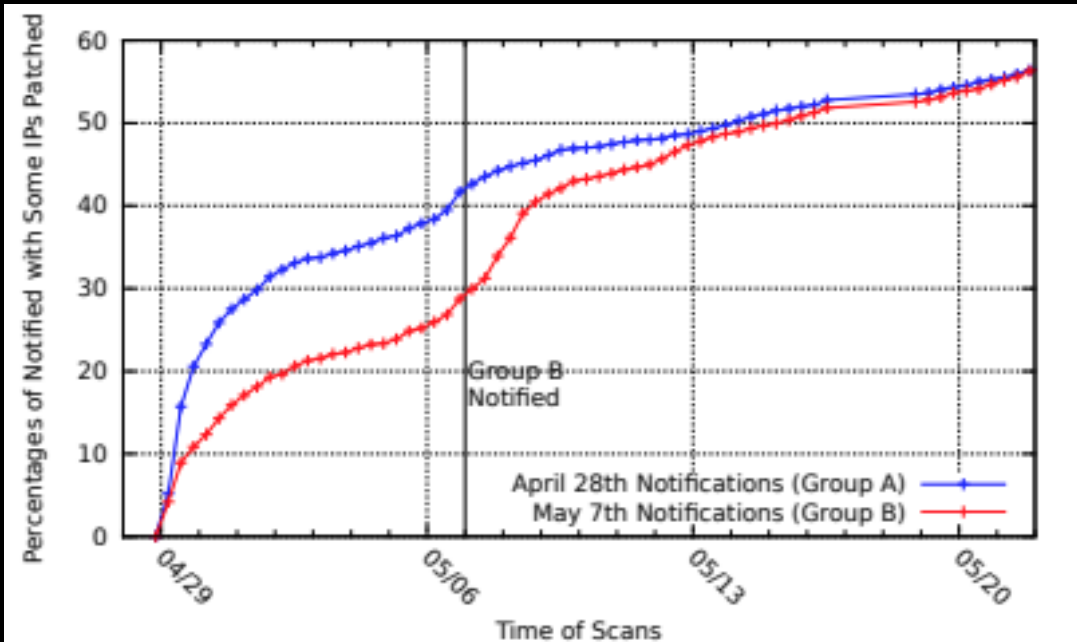


Figure 9: **Patch Rates of Group A vs Group B.** The patch rates for our two notification sets show that notification had statistically significant impact on patch rate.

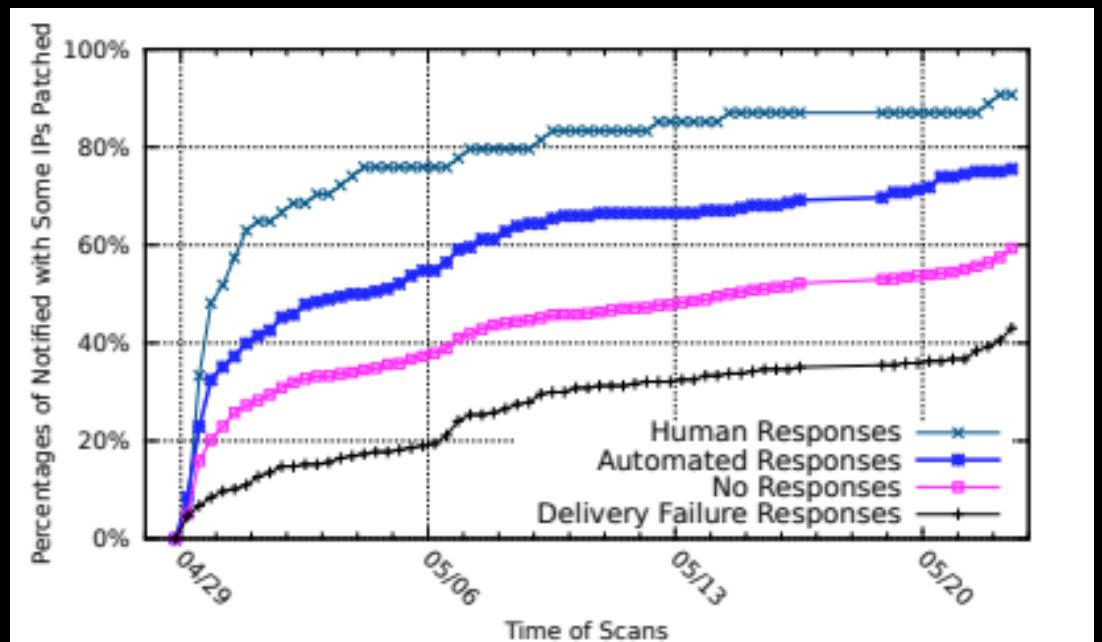
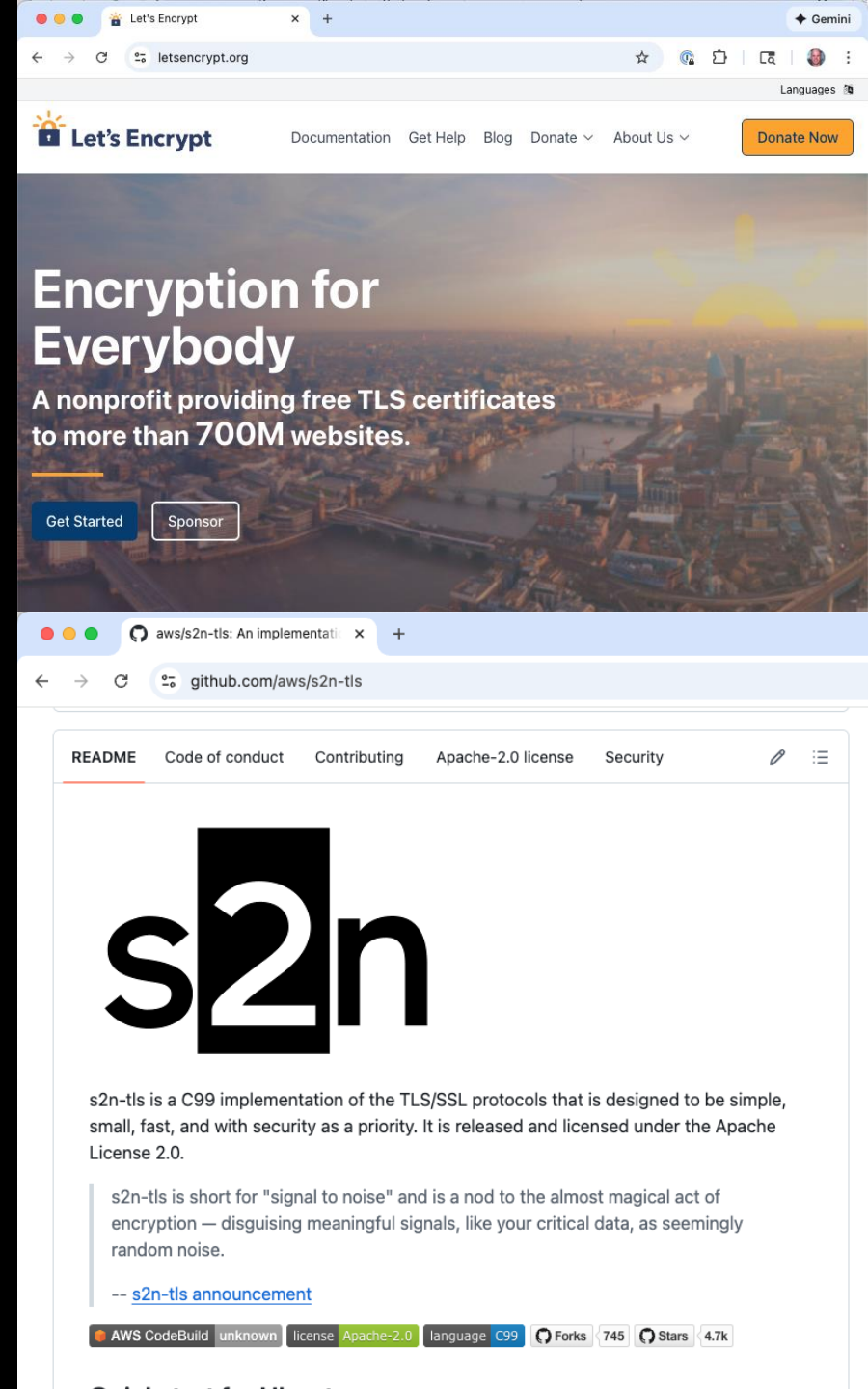


Figure 10: **Patch Rates for Different Response Types.** Conditioning on the sort of reply we received for a given notification reveals statistically significant differences.

Notes

- Forward secrecy: Heartbleed highlights its importance
 - But: only 44% of the connections observed by the ICSI Notary in May 2014 used it
- Then, certificate management mostly manual
 - Today: Let's Encrypt!
- Then: Only 45% of Alexa top 1M used HTTPS
 - In 2019: 71.67% (couldn't find more recent)
- OpenSSL = common point of trust
 - E.g., galvanized AWS' s2n TLS implementation
- Whole affair highlighted the need for a robust notification system



Discussion

- What do you think about the outcomes of this study?
 - Have learned something worthwhile?
 - Did the study, along with the event, precipitate positive change?
- What do you think about the methodology of the study?
 - Are the methods sound (enough)?
 - Is this the way Internet-wide assessments are best done?
- What other sorts of questions might we have liked the answers to?